A Report on

Linux Performance Analyzer Using PROC file system

BACHELOR OF TECHNOLOGY
in
INFORMATION TECHNOLOGY

by

Kethavath Santhosh Naik - 241IT038

under the guidance of
Dr Janani T, Department of IT

Department of Information Technology
National Institute of Technology Karnataka, Surathkal.

APRIL 2026

Abstract

The Linux Performance Analyzer is a system-level monitoring tool developed in C that directly interfaces with the Linux kernel through the /proc virtual filesystem and /sys pseudo-filesystem. The tool provides real-time visibility into CPU usage, memory pressure, per-process resource consumption, disk I/O, network statistics, file descriptor usage, and battery status — without any external libraries or dependencies.

Unlike traditional utilities such as top or htop, this analyzer exposes the raw kernel data structures and counter values underlying those tools, giving users direct insight into how the operating system tracks and reports system health. The project implements an intelligent three-level alert mechanism (WARNING / CRITICAL / EMERGENCY) with persistent logging, per-core performance tracking with delta sampling, per-process CPU percentage calculation using scheduler tick deltas, and automated snapshot export for incident response.

The tool is architecturally equivalent to the data collection layer of enterprise monitoring platforms such as Prometheus node_exporter, Datadog Agent, and Nagios NRPE — demonstrating practical knowledge of Linux kernel internals, systems programming, and production monitoring principles.

Project Overview

This project builds a command-line Linux performance analyzer entirely in C, reading live kernel data through the /proc and /sys virtual filesystems. The tool monitors the same kernel counters that enterprise monitoring products rely on, exposing them in a human-readable, color-coded terminal interface with automated alerting and logging.

The analyzer is designed to demonstrate deep operating system knowledge — specifically how the Linux kernel maintains internal data structures for CPU scheduling, memory management, process lifecycle, and I/O accounting, and how those structures are surfaced through the procfs interface.

Objectives

- Read and interpret live kernel data directly from /proc and /sys without external libraries.
- Implement real-time CPU usage calculation using scheduler tick counter deltas.
- Track per-process CPU percentage and memory footprint with two-snapshot sampling.
- Monitor per-core utilization independently to detect thread hotspots.
- Implement a three-level intelligent alert system (WARNING / CRITICAL / EMERGENCY) with file logging.
- Parse TCP connection states from /proc/net/tcp to detect connection leaks.
- Monitor disk usage and I/O throughput using /proc/diskstats and statvfs().
- Track file descriptor exhaustion — a common production outage cause.
- Monitor battery status on laptops via /sys/class/power_supply/.
- Export complete system snapshots to timestamped files for incident response.

System Architecture

The tool is structured as a single-file C program with clearly separated modules, each responsible for one subsystem. The architecture follows a three-layer design:

Layer 1 — Kernel Data Layer

The Linux kernel maintains live data structures in memory: kernel_cpustat[] for CPU counters, task_struct for each process, zone-based page allocator counters for memory, and network socket tables. These are not stored on disk — they exist purely in kernel RAM.

Layer 2 — procfs / sysfs Interface Layer

The kernel exposes these structures through two virtual filesystems: /proc (process and system info) and /sys (device and hardware info). When user-space opens a file like /proc/stat, the kernel's procfs handler reads the relevant data structure and formats it as ASCII text on demand. No disk I/O is involved.

Layer 3 — Analyzer (This Program)

The analyzer opens these virtual files using standard C file I/O (fopen/fscanf), parses the ASCII output, performs calculations (delta CPU%, memory percentages, etc.), applies alert thresholds, and presents results to the user with color-coded terminal output.

Your Program (User Space)	procfs / sysfs	Kernel Data Structures
fopen() / fscanf()	Formats on read	kernel_cpustat[]
calculate_cpu_usage()	/proc/stat	scheduler tick counters
show_memory()	/proc/meminfo	NR_FREE_PAGES, zone counters
show_process_stats()	/proc/[pid]/stat	task_struct.__state
show_network()	/proc/net/dev, tcp	socket hash tables
show_battery()	/sys/class/power_supply/	ACPI battery driver

The /proc Filesystem — Kernel Internals

Understanding /proc at the kernel level is the theoretical foundation of this project. The following explains what actually happens inside the OS when the analyzer reads each file.

What is /proc?

/proc is a virtual filesystem (type: procfs) mounted at /proc. It contains no real files — every entry is generated dynamically by the kernel when read. It was created to give user-space programs a standardized interface to kernel internal data without requiring privileged system calls for every query.

How /proc Works Internally

When the kernel boots, it calls proc_init() which registers procfs with the VFS (Virtual File System) layer. When user-space opens /proc/stat, the call chain is:

```
fopen("/proc/stat")  →  sys_open()  →  VFS lookup
  →  procfs: proc_lookup()  →  proc_stat_operations
  →  on read: show_stat()  [fs/proc/stat.c]
  →  reads: kernel_cpustat[cpu]  (per-CPU counters)
  →  formats via seq_file interface
  →  returns ASCII to your fscanf()
```

There is zero disk I/O. The data is read directly from kernel memory and formatted on-the-fly every time the file is opened.

Key Kernel Source Files

Kernel File	What it Contains
kernel/fork.c	copy_process(), kernel_clone() — process creation
kernel/exit.c	do_exit(), release_task() — process termination
fs/proc/base.c	Per-PID /proc entries, proc_pid_lookup()
fs/proc/stat.c	Generates /proc/stat from kernel_cpustat[]
fs/proc/meminfo.c	Generates /proc/meminfo from page allocator
fs/seq_file.c	seq_file: kernel helper for /proc text generation
include/linux/sched.h	task_struct definition — core process descriptor
kernel/pid.c	PID allocation, namespaces, alloc_pid()

task_struct — The Process Descriptor

Every process in Linux is represented in the kernel as a task_struct (defined in include/linux/sched.h). When the analyzer reads /proc/[pid]/stat, the kernel directly reads fields from that process's task_struct and formats them as text:

task_struct Field	/proc/pid/stat Field	What It Means
task->pid	Field 1	Process ID
task->comm	Field 2	Process name
task->__state	Field 3 (R/S/Z/D/T)	Current run state

task_struct Field	/proc/pid/stat Field	What It Means
task->real_parent->pid	Field 4	Parent PID
task->utime	Field 14	User CPU time (jiffies)
task->stime	Field 15	Kernel CPU time (jiffies)
task->mm->total_vm	Field 23	Virtual memory size

Module-by-Module Description

6.1 CPU Details & Events

Source: /proc/stat | Function: show_cpu_details()

The kernel updates CPU time counters on every scheduler tick (typically 250 or 1000 Hz). The aggregate cpu line in /proc/stat is read from kernel_cpustat[], which stores cumulative jiffies since boot in eight categories:

Field	Kernel Counter	Meaning
user	CPUTIME_USER	Normal user-mode execution
nice	CPUTIME_NICE	Low-priority user processes
system	CPUTIME_SYSTEM	Kernel mode execution
idle	CPUTIME_IDLE	CPU doing nothing
iowait	CPUTIME_IOWAIT	Waiting for disk I/O
irq	CPUTIME_IRQ	Handling hardware interrupts
softirq	CPUTIME_SOFTIRQ	Software interrupt processing
steal	CPUTIME_STEAL	Time stolen by hypervisor (VMs)

CPU usage formula: usage% = (total_diff − idle_diff) / total_diff × 100

Two readings are taken 1 second apart. The difference gives activity in that window — identical to how top calculates its %CPU column.

Context switches (ctxt), total interrupts (intr), processes forked since boot, and currently running/blocked process counts are also read from /proc/stat.

6.2 Per-Core Performance Tracking with Logging

Source: /proc/stat (cpu0, cpu1 … lines) | Function: show_percore_tracking()

Reads each per-core cpu[N] line from /proc/stat twice with a 1-second interval. Calculates delta for user%, sys%, iowait%, and idle% independently per core. Results are logged to /tmp/analyzer_percore.log with a timestamp. If any single core exceeds 85% usage, a CRITICAL hotspot alert is fired and logged — enabling detection of thread imbalance where one core is saturated while others are idle.

6.3 Per-Process Resource Tracking

Source: /proc/[pid]/stat, /proc/[pid]/status | Function: show_process_resource_tracking()

This is the most technically involved module. It calculates real CPU% per process using the same delta technique used by top:

```
1. Read total system ticks  (get_cpu_times)       ← snapshot 1
2. Read utime+stime for all PIDs                  ← snapshot 1
3. sleep(1)
4. Read total system ticks  (get_cpu_times)       ← snapshot 2
5. Read utime+stime for all PIDs                  ← snapshot 2

proc_delta  = (utime2 - utime1) + (stime2 - stime1)
sys_delta   = total_ticks2 - total_ticks1
cpu_percent = proc_delta / sys_delta * 100
```

Memory percentage is calculated as: vmrss / MemTotal * 100, where vmrss (physical RAM in KB) is read from the VmRSS field in /proc/[pid]/status. Any process using more than 20% CPU or 500 MB RAM triggers an alert logged to /tmp/analyzer_procs.log.

6.4 Memory Monitoring

Source: /proc/meminfo | Function: show_memory()

The kernel maintains per-zone page counters that are updated atomically every time a page is allocated or freed. /proc/meminfo is generated by meminfo_proc_show() in fs/proc/meminfo.c, reading these counters:

Field	Kernel Source	Industry Significance
MemTotal	totalram_pages() at boot	Total physical RAM
MemFree	NR_FREE_PAGES zone counter	Completely unused pages
MemAvailable	si_mem_available()	What apps can actually use (free + reclaimable)
Dirty	NR_FILE_DIRTY counter	Pages modified but not yet flushed to disk
SwapTotal/Free	total_swap_pages	Swap utilisation — indicates memory pressure
AnonPages	NR_ANON_MAPPED	Anonymous memory (heap, stack, mmap)

6.5 Load Average

Source: /proc/loadavg | Function: show_loadavg()

The kernel calculates load average in kernel/sched/loadavg.c using calc_global_load(), called every 5 seconds. It applies an exponential moving average formula with decay constants for 1, 5, and 15 minute windows. The result is stored in the avenrun[3] array. Load average represents the average number of processes in a runnable or uninterruptible state — not CPU percentage. A load above the core count means the system has more work than CPU resources.

6.6 Network Interfaces & TCP States

Sources: /proc/net/dev, /proc/net/tcp | Function: show_network()

/proc/net/dev provides per-interface counters for bytes, packets, and errors for both receive (RX) and transmit (TX) directions. These counters live in the kernel's net_device structure.

/proc/net/tcp lists all TCP sockets in the system with their hex-encoded state values. The analyzer decodes these into human-readable states:

Hex	State	Industry Meaning
0x01	ESTABLISHED	Active connection — normal
0x0A	LISTEN	Server waiting for connections
0x06	TIME_WAIT	Connection closed, waiting for late packets. High count = port exhaustion risk
0x08	CLOSE_WAIT	Remote closed, local not. High count = connection leak bug in application

6.7 Disk Usage & I/O Statistics

Sources: /proc/mounts + statvfs(), /proc/diskstats | Function: show_disk()

/proc/mounts lists all currently mounted filesystems. For each real filesystem, statvfs() is called to get block counts (total, free, available). /proc/diskstats provides per-device I/O counters (reads, writes, sectors transferred) from the kernel's block layer. Virtual filesystems (proc, sysfs, tmpfs, cgroup, overlay) are filtered out to show only real storage devices.

6.8 File Descriptors & Kernel Limits

Source: /proc/sys/fs/file-nr | Function: show_fd_info()

/proc/sys/fs/file-nr returns three values: allocated FDs, unused FDs, and the system maximum. The analyzer calculates used FDs = allocated - unused and shows a usage percentage bar. FD exhaustion (the 'too many open files' error) is one of the most common production outages in Node.js, Java, and Python servers — this module detects it before it causes failures. Additional kernel tuning parameters (pid_max, threads-max, vm.swappiness, ip_forward) are also displayed.

6.9 Battery Usage Monitoring

Source: /sys/class/power_supply/BAT0/ | Function: show_battery()

Battery information is exposed through the Linux power supply subsystem in /sys/class/power_supply/. The analyzer reads capacity, charge status, energy_now, energy_full, voltage_now, technology, and cycle_count. From energy_now and power_now it calculates estimated remaining runtime. Intelligent alerts fire at 20% (WARNING) and 10% (CRITICAL), both printed and logged.

6.10 Real-Time Monitor

Function: realtime_monitor()

Takes 5 consecutive snapshots of CPU usage, memory usage, and 1-minute load average at 2-second intervals. Each snapshot is timestamped and logged to the alert log. Intelligent 3-level alerts are evaluated on every snapshot. This simulates the behavior of a monitoring agent like the Prometheus node_exporter, which scrapes system metrics at a fixed interval and evaluates alerting rules on each scrape.

Intelligent Alert Mechanism

The analyzer implements a three-level severity alert system via the intelligent_alert() function. This mirrors the alerting architecture used in Prometheus Alertmanager, PagerDuty, and AWS CloudWatch.

Level	CPU Threshold	Memory Threshold	Color	Action
OK	< 60%	< 70%	Green	No alert
WARNING	≥ 60%	≥ 70%	Yellow	Print + log
CRITICAL	≥ 85%	≥ 85%	Red	Print + log
EMERGENCY	≥ 95%	≥ 95%	Red BG	Print + log

Additional alerts are fired for:

- **Process CPU Alert:** Single process using > 20% CPU
- **Process Memory Alert:** Single process using > 500 MB RAM
- **Battery Alert:** Battery below 20% (WARNING) or 10% (CRITICAL)
- **Disk Alert:** Disk partition above 90% usage
- **FD Alert:** File descriptor usage above 80%
- **TCP Alert:** TCP TIME_WAIT count above 100 or CLOSE_WAIT above 50
- **Per-Core Alert:** Any single CPU core above 85% (hotspot alert)

Logging System

Every alert, monitoring snapshot, and session event is written to one of three log files using the write_log() function, which prepends an ISO-format timestamp and severity tag to every entry.

Log File	Tag	Contents
/tmp/analyzer_alerts.log	WARNING/CRITICAL/EMERGENCY/ALERT/SESSION/MONITOR/SNAPSHOT/BATTERY	All system-level alerts, real-time monitor snapshots, session start/end
/tmp/analyzer_percore.log	CORE-ALERT	Per-core CPU breakdown (user/sys/iowait/idle%) per snapshot, core hotspot alerts
/tmp/analyzer_procs.log	PROC-CPU / PROC-MEM	Top process CPU/memory usage per snapshot, high-resource process alerts

Log format example:

```
[Wed Mar 15 14:30:22 2024] [CRITICAL] CRITICAL: CPU=87.3% exceeded threshold 85.0%
[Wed Mar 15 14:30:24 2024] [PROC-CPU] HIGH CPU: PID=1234 name=python3 cpu=23.4%
[Wed Mar 15 14:32:00 2024] [SNAPSHOT] /tmp/sys_snapshot_20240315_143200.txt
```

Snapshot Export

The export_snapshot() function redirects stdout to a timestamped file and calls all monitoring functions sequentially, capturing complete system state at one moment in time. The file is named using the format:

```
/tmp/sys_snapshot_YYYYMMDD_HHMMSS.txt
```

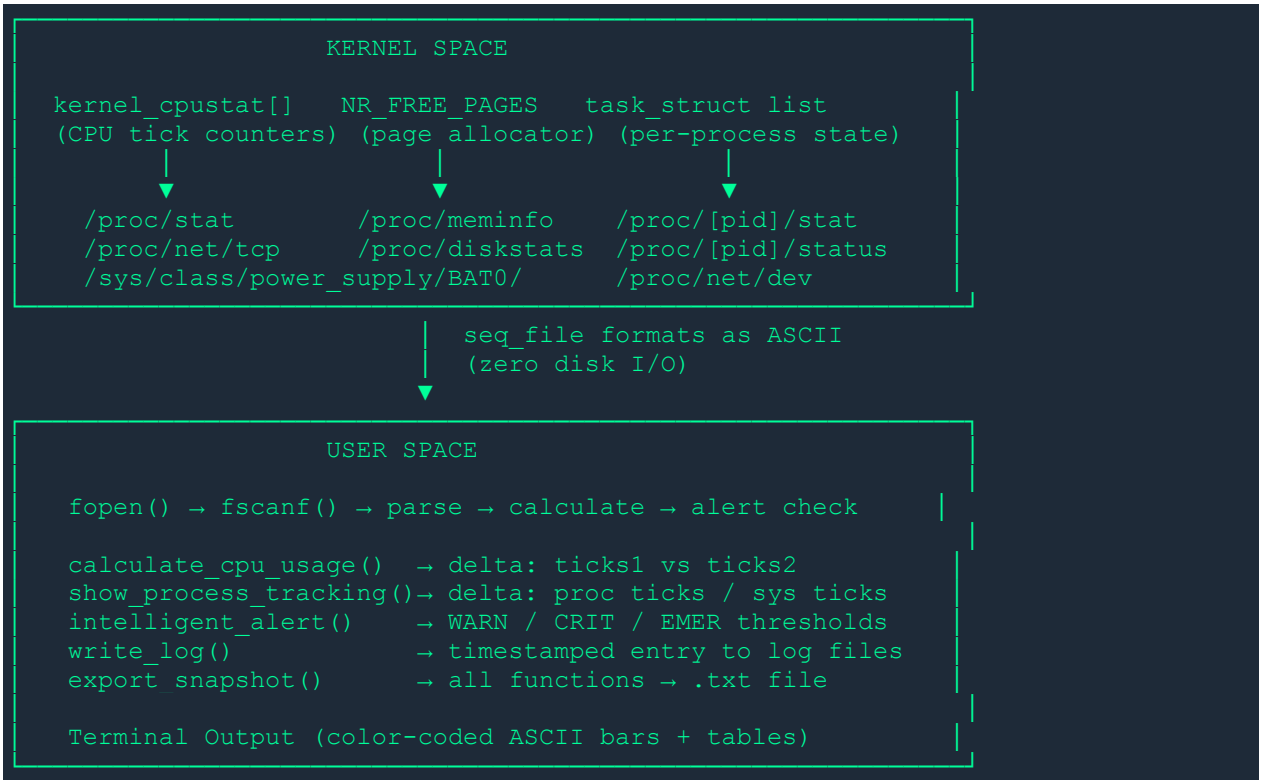
Snapshots are used in industry for:

- Post-mortem analysis after server crashes — examining system state before the failure
- Before/after deployment comparison — verifying a new release does not degrade performance
- Incident sharing — sending a single file to senior engineers for remote diagnosis
- Compliance auditing — proving system health at a specific point in time

This is equivalent to the sosreport tool on Red Hat Enterprise Linux and the sysdig capture facility, both of which are standard tools in enterprise DevOps and SRE workflows.

Data Flow Diagram

The following diagram shows how data flows from kernel memory to the user's terminal:



Key Kernel Data Structures

task_struct (include/linux/sched.h)

The master data structure for every process. Contains hundreds of fields including process state, scheduling info, memory descriptor, file table, signal handlers, credentials, CPU time accounting, and namespace references. /proc/[pid]/stat and /proc/[pid]/status are direct views into this structure.

kernel_cpustat (include/linux/kernel_stat.h)

A per-CPU array of time counters maintained by the scheduler. Updated on every timer interrupt. The aggregate cpu line in /proc/stat is the sum across all CPUs. The per-core cpu[N] lines come from individual array entries.

mm_struct (include/linux/mm_types.h)

The memory descriptor attached to each process's task_struct. Contains the page table, virtual memory area list, and memory usage statistics including total_vm, rss_stat (which drives VmRSS in /proc/pid/status), and file-backed vs anonymous mapping counts.

inet_sock / tcp_sock (include/net/inet_sock.h)

Kernel structures representing TCP sockets. The state field maps directly to the hex values in /proc/net/tcp. The analyzer decodes ESTABLISHED (0x01), LISTEN (0x0A), TIME_WAIT (0x06), and CLOSE_WAIT (0x08) from these values.

Structure	Lives In	Exposed Via
task_struct	kernel slab allocator (heap)	/proc/[pid]/stat, /proc/[pid]/status
kernel_cpustat	per-CPU data section	/proc/stat
mm_struct	attached to task_struct	/proc/[pid]/status (VmRSS etc.)
zone page counters	struct zone in kernel	/proc/meminfo
inet_sock / tcp_sock	socket slab cache	/proc/net/tcp
gendisk / request_queue	block layer	/proc/diskstats

Industry Relevance

The following table maps each feature of this project to its equivalent in enterprise monitoring tools used at companies like Google, Meta, Netflix, Amazon, and Flipkart:

This Project's Feature	Industry Tool Equivalent	Used At
CPU delta calculation from /proc/stat	Prometheus node_exporter node_cpu_seconds_total	All cloud companies
Per-process CPU% tracking	top, htop, ps, pidstat	Every Linux sysadmin
Three-level alert thresholds	Prometheus Alertmanager rules, PagerDuty severity levels	Google, Amazon, Netflix
TCP state tracking	ss -s, netstat, Datadog network monitor	Web servers, microservices
Disk I/O from /proc/diskstats	iostat, sar, Prometheus node_disk_*	Database admins (MySQL, PostgreSQL)
FD exhaustion monitoring	Custom alerts in Node.js/Java apps	All production server teams
Timestamped snapshot export	sosreport (RHEL), sysdig capture	Red Hat, enterprise Linux support
Persistent alert logging	ELK Stack, Splunk, CloudWatch Logs	AWS, GCP, Azure customers
Real-time periodic sampling	Prometheus scrape interval, Telegraf	All observability platforms

How to Compile & Run

Requirements

- Linux operating system (kernel 3.x or later)
- GCC compiler
- No external libraries required — uses only standard C and POSIX

Compile

```
gcc -o analyzer linux_analyzer_v3.c -Wall

# Or with optimization:
gcc -O2 -o analyzer linux_analyzer_v3.c -Wall
```

Run

```
./analyzer
```

Menu Options

#	Feature	Log File
1	CPU Details + Context Switches + Interrupts	/tmp/analyzer_alerts.log
2	Per-Core Performance Tracking with Logging	/tmp/analyzer_percore.log
3	Per-Process Resource Tracking (CPU% + MEM%)	/tmp/analyzer_procs.log
4	System Uptime	—
5	Load Average with Core Comparison	—
6	Memory — Extended + 3-Level Alerts	/tmp/analyzer_alerts.log
7	Process Analysis — All States	—
8	Top 10 Processes by Memory & CPU Time	—
9	Network Interfaces + TCP Connection States	/tmp/analyzer_alerts.log
10	Disk Usage + I/O Statistics	/tmp/analyzer_alerts.log
11	File Descriptors & Kernel Limits	/tmp/analyzer_alerts.log
12	Battery Usage Monitoring	/tmp/analyzer_alerts.log
13	Real-Time Monitor (5 snapshots × 2s)	/tmp/analyzer_alerts.log
14	View All Alert & Event Logs	—
15	Export Full Snapshot to Timestamped File	/tmp/sys_snapshot_*.txt
16	Show ALL Information	all logs

Conclusion

The Linux Performance Analyzer demonstrates end-to-end mastery of Linux system programming — from the kernel data structures that track process state and CPU accounting, through the procfs virtual filesystem that exposes them to user space, to the C code that reads, parses, and acts on that data.

The project goes beyond a simple display tool by implementing production-quality features: intelligent multi-level alerting, persistent structured logging across three log files, per-process real CPU percentage calculation using scheduler tick deltas, per-core hotspot detection, battery monitoring with estimated runtime, and incident-response snapshot export.

These capabilities are architecturally equivalent to the data collection and alerting layers of enterprise monitoring platforms, making this project a strong demonstration of both theoretical OS knowledge and practical systems engineering judgment.

Skill Demonstrated	Evidence in Code
Linux kernel internals	Reads task_struct fields, kernel_cpustat[], zone page counters via procfs
Systems programming in C	Manual parsing, pointer arithmetic, struct-based data model, file I/O
OS concepts	Scheduler ticks, memory zones, TCP state machine, process lifecycle
Production engineering thinking	3-level alerts, persistent logs, snapshot export, FD exhaustion detection
Tool awareness	Implements what top, htop, iostat, ss, Prometheus node_exporter do internally